

合肥工业大学

C 语言程序设计上  
机实验指导书

2024 年 4 月

# 前言 上机实验的目的和要求

## 一 上机实验的目的

上机实验的目的，绝不仅仅是为了验证教材和讲课的内容，或者验证自己所编程序正确与否。学习程序语言，上机实验的目的如下：

- 1 加深对讲授内容的理解，尤其是一些语法规则，通过多次上机，熟练地掌握语法。
- 2 了解和熟悉 C 语言程序开发环境。熟悉一两种环境，再遇到其他的系统时便会触类旁通。
- 3 学会上机调试程序，也就是善于发现程序中的错误，并且能很快地排除这些错误，使程序能够正确地运行。要真正掌握计算机应用技术，就不仅应当了解和熟悉有关理论和方法，而且要求自己动手实践能力强。
- 4 在做实验时千万不要在程序通过后就认为万事大吉，完成任务了，应当在通过的程序上做一些调试和修改，看看会得到什么结果。

## 二 上机实验前的准备工作

- 1 了解所用的计算机系统的性能和使用方法；
- 2 复习和掌握与本实验有关的教学内容；
- 3 准备好上机所用的程序，切忌自己不思考、抄袭别人的程序；
- 4 准备好调试和运行时所需的代码。

## 三 上机实验的步骤

- 1 进入 C 语言工作环境；
- 2 输入自己编制好的程序；
- 3 检查输入是否有错，及时更正；
- 4 进行编译和链接；
- 5 运行程序，分析结果。

## 四、实验结束，整理实验报告

实验报告分两份，实验一到实验四一份，实验六到实验十一份

应包括以下内容：

- 1 上机题目；
- 2 程序清单；
- 3 运行结果；
- 4 对结果的分析和本本次获得的经验和体会。

# 实验一 C 语言集成开发环境与使用指南

## 一 实验目的

1. 认识和理解集成开发环境的作用和重要性。包括集成开发环境能够提供的编程支持，比如代码编辑、编译、调试以及项目管理等功能。
2. 学习如何使用 Code::Blocks 这一特定集成开发环境进行 C/C++ 的程序开发。
3. 熟悉 Code::Blocks 中使用的编译器，如 GCC 和 MSVC++。
4. 指引学生通过实践活动理解理论知识，掌握实际技能，为后续的编程实践打下基础。

## 二 实验内容

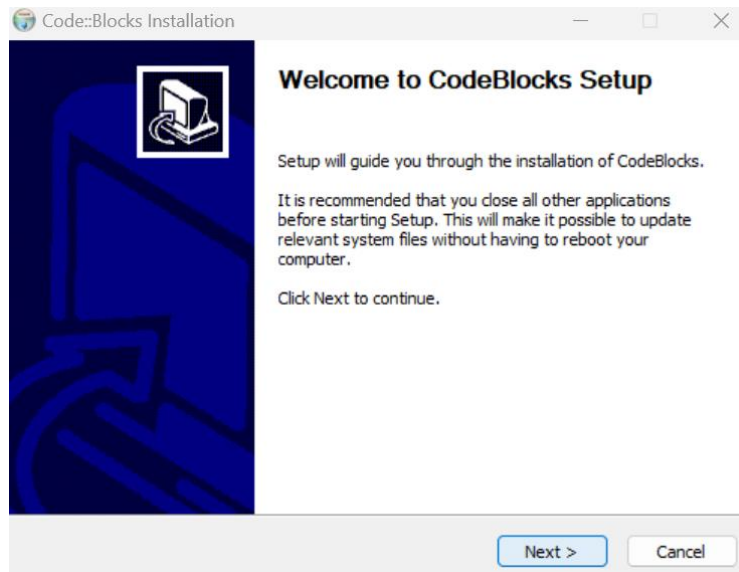
1. Code::Blocks
  - 1) 免费且开源：Code::Blocks 是免费提供的，并且它的源代码是公开的，这意味着任何人都可以下载源代码并根据自己的需要修改它。
  - 2) 体积小巧：相较于一些其他的 IDE，Code::Blocks 的安装包较小，这使得下载和安装过程快速且不占用太多磁盘空间。
  - 3) 安装简单：不需要复杂的配置，安装过程中也不需要设置特定的环境变量。它提供了易于使用的安装程序，适用于不同的操作系统。
2. 下载安装
  - 1) 直接从[官网](#)下载



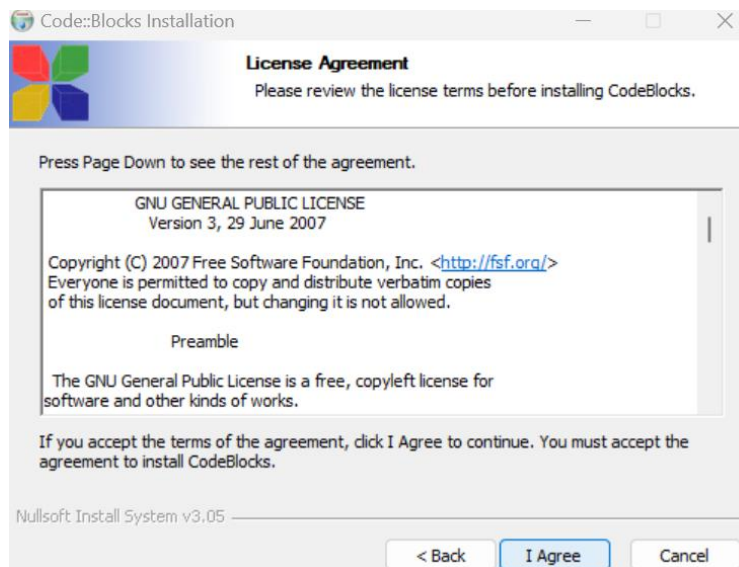
### Microsoft Windows

| File                                      | Download from                                              |
|-------------------------------------------|------------------------------------------------------------|
| codeblocks-20.03-setup.exe                | <a href="#">FossHUB</a> or <a href="#">Sourceforge.net</a> |
| codeblocks-20.03-setup-nonadmin.exe       | <a href="#">FossHUB</a> or <a href="#">Sourceforge.net</a> |
| codeblocks-20.03-nosetup.zip              | <a href="#">FossHUB</a> or <a href="#">Sourceforge.net</a> |
| codeblocks-20.03mingw-setup.exe           | <a href="#">FossHUB</a> or <a href="#">Sourceforge.net</a> |
| codeblocks-20.03mingw-nosetup.zip         | <a href="#">FossHUB</a> or <a href="#">Sourceforge.net</a> |
| codeblocks-20.03-32bit-setup.exe          | <a href="#">FossHUB</a> or <a href="#">Sourceforge.net</a> |
| codeblocks-20.03-32bit-setup-nonadmin.exe | <a href="#">FossHUB</a> or <a href="#">Sourceforge.net</a> |
| codeblocks-20.03-32bit-nosetup.zip        | <a href="#">FossHUB</a> or <a href="#">Sourceforge.net</a> |
| codeblocks-20.03mingw-32bit-setup.exe     | <a href="#">FossHUB</a> or <a href="#">Sourceforge.net</a> |
| codeblocks-20.03mingw-32bit-nosetup.zip   | <a href="#">FossHUB</a> or <a href="#">Sourceforge.net</a> |

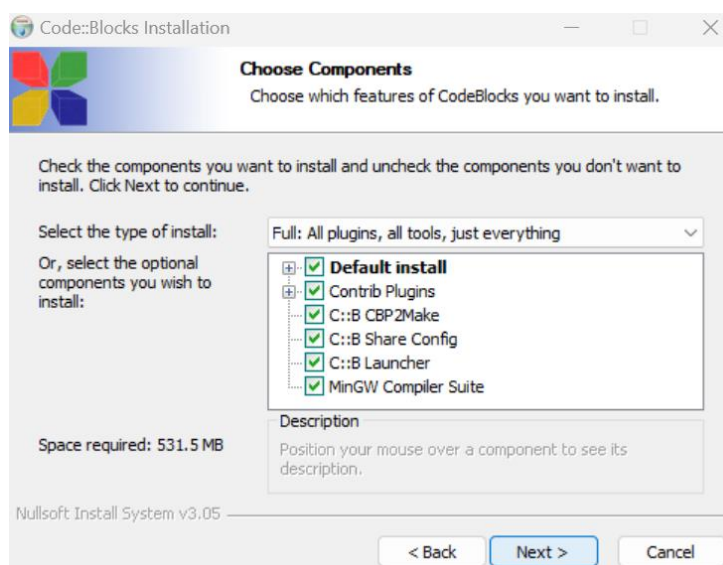
- 2) 推荐下载< codeblocks-20.03mingw-setup.exe >，点击第 4 行的 FossHUB，页面会自动下载<codeblocks-20.03mingw-setup.exe>，大小是 145MB。
- 3) 运行 codeblocks-20.03mingw-setup.exe。
  - a) 点击“Next >”



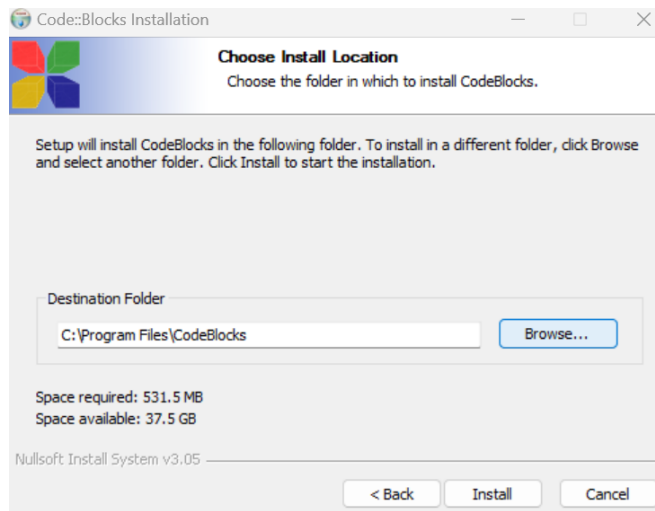
b) 点击“I Agree”



c) 安装全部组件，点击“Next >”



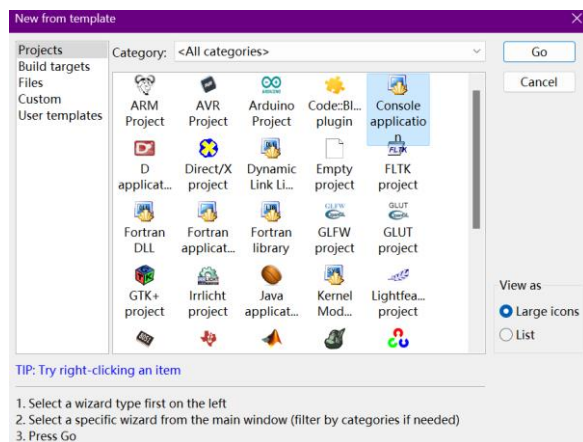
- 4) 选择安装地址(CodeBlocks 装在了除 C 盘外的其他盘中，导致系统无法正确识别出编译器的地址。在安装时将软件装在默认地址（推荐），或者手动更新编译器所在的路径)。



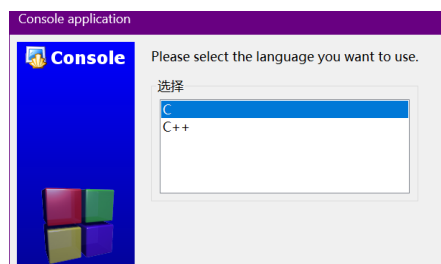
- 5) 点击“Install”，安装即可。

### 3. 运行 Code::Blocks

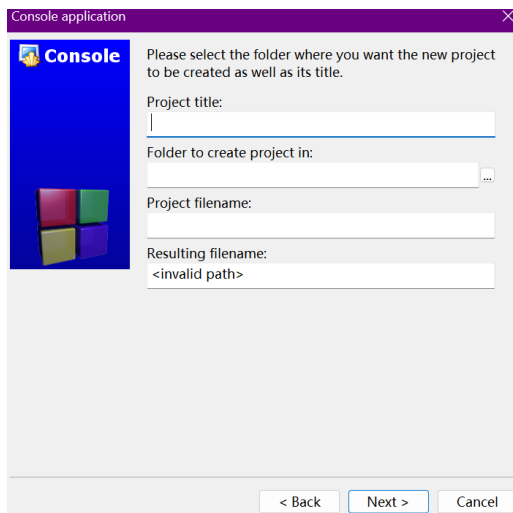
- 1) 双击开始运行。
- 2) 创建新项目（create a new project）。
- 3) 选择 Console Application（控制台应用程序）。



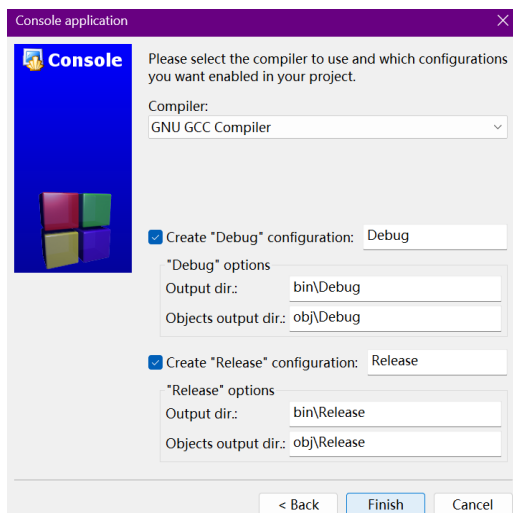
- 4) 选择 C 项目。



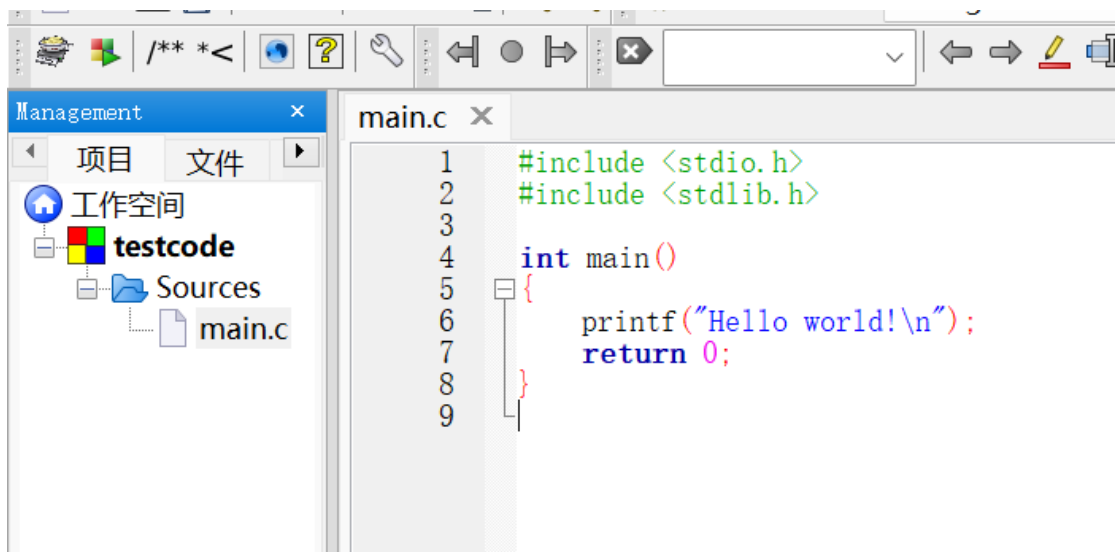
- 5) 创建文件名地址，创建的项目需要在英文路径下，不能包含中文字符。



- 6) 编译器和配置文件，默认即可，“Finish”。

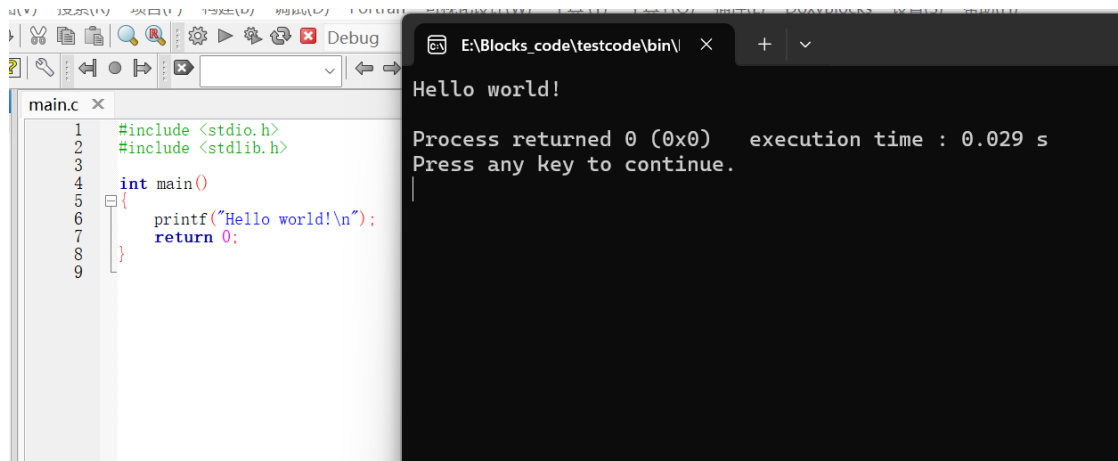


- 7) 左侧管理器，展开项目双击 main.c 可得到样例代码 hello world



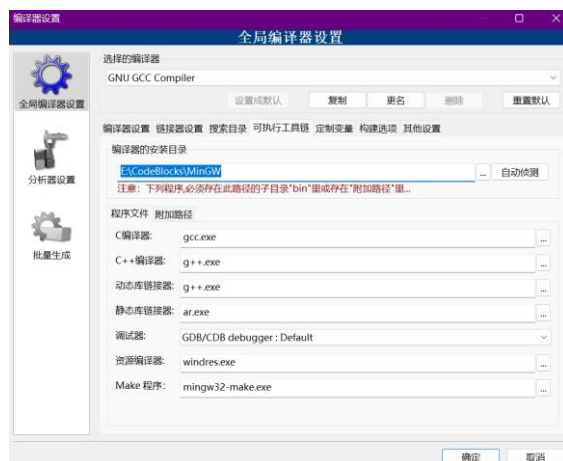
- 8) 点击构建并运行可得到运行结果（也可先构建再运行）





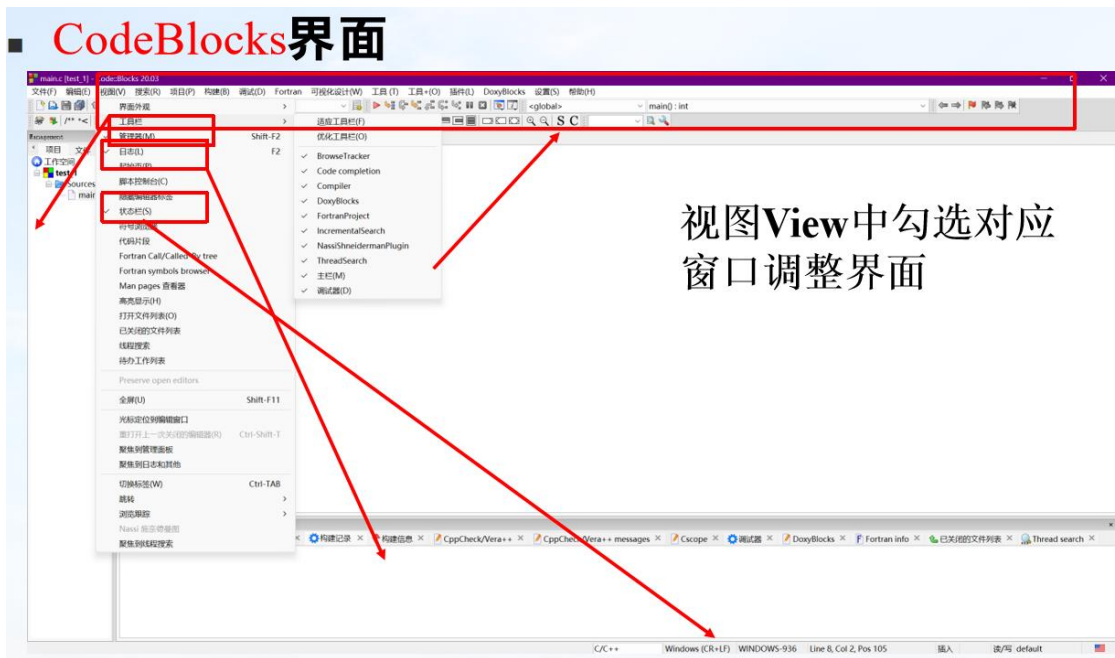
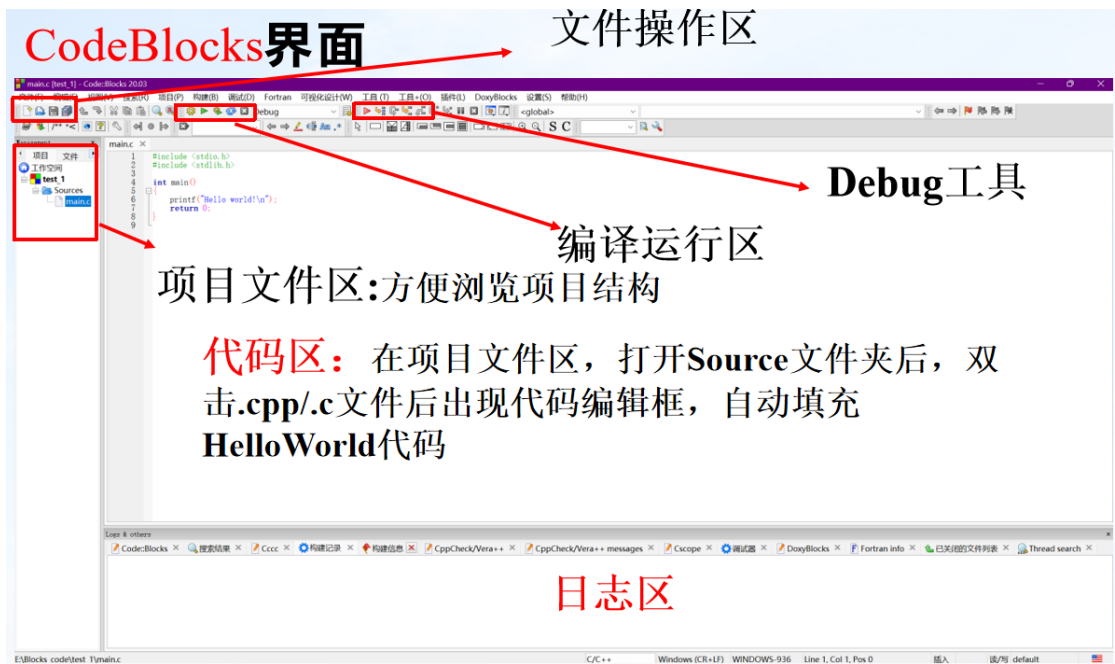
4. 更改编译器地址（安装地址不是默认地址，需要手动更新编译器所在的路径；安装地址为默认地址可忽略）

1) 设置-->编译器，即 Setting --> Compiler; Toolchain exectables，即可执行的工具链接



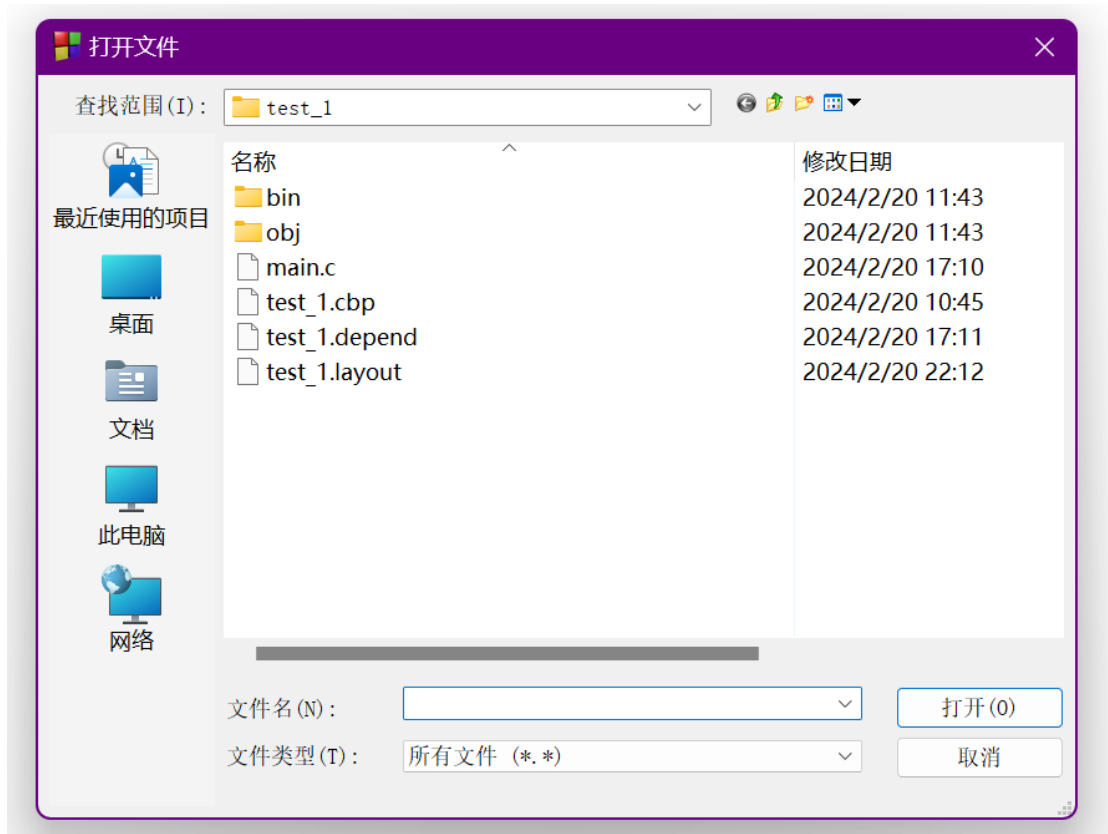
2) 点击图标 "...", 找到你的安装目录，在你所选的目录下应该有个 bin(工具箱)文件夹，并且所有的编译器应该都在这个 bin 目录下

5. CodeBlocks 界面



6. 打开已有文件

1) 点击 File->Open



2) 在新弹出的窗口内选择`**cbp`的工程文件，就可以打开该工程文件

## 7. CodeBlocks 编译运行

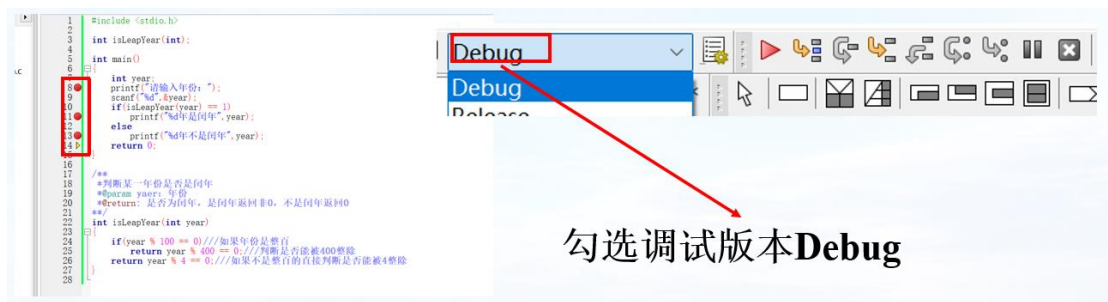
- 1) 编译：依赖于编译器 `compile`, 将源码转化成目标文件，如`.obj`文件
- 2) 生成：链接过程 `build`，依赖于链接器。将所有的目标文件和所有需要用到的组建组合成一个整体
- 3) 调试：让程序在系统中运行之前的查错和改错；让程序在调试器中运行，用这种手段对程序进行分析，找出并修正潜在的问题
- 4) 运行：让程序在系统中运行
- 5) `Build`、`Run`、`Build&Run`、`ReBuild`、`Abort` 我们要运行一个项目，应该先 `build` 再 `run`，或者 `Build&Run` (`F9`)



## 8. CodeBlocks 调试

- 1) 调试器：跟踪程序执行过程，调试问题代码，程序逻辑复杂时候，使用调试器可以极大地提高程序的优化和维护效率

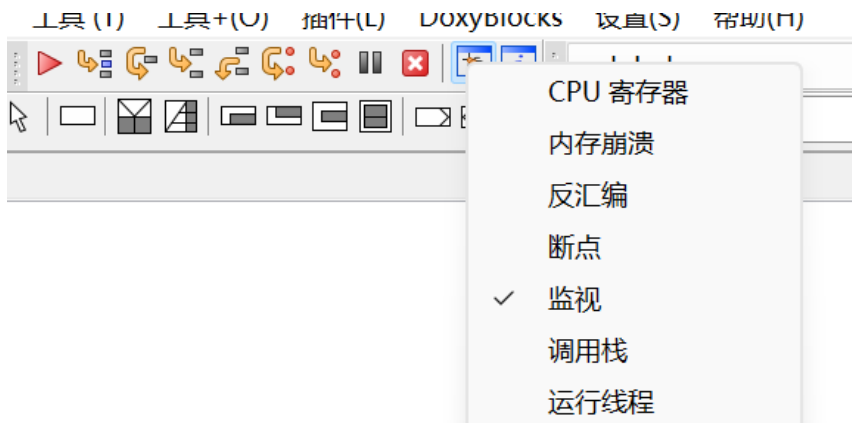
- 2) 断点:让程序在需要的位置中断,在启动调试器前设置断点如下,鼠标点击编辑器的左边即可设置断点,再次单击取消

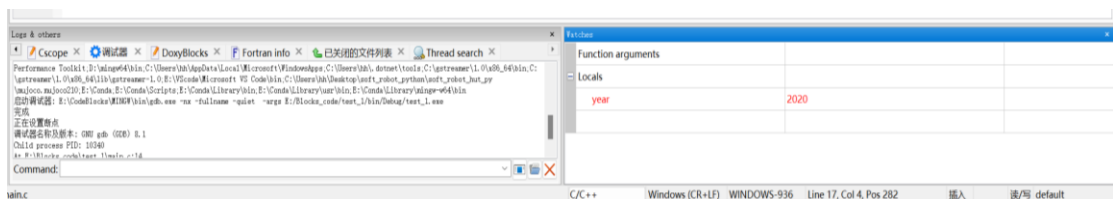


- 3) 调试:并且运行到断点处暂停运行,此时它叫 Debug,如果在调试过程中点击此按钮,则会一直运行到下一个断点处停下
- 4) 执行至光标处:将光标定位到目标行,编译器会自动运行至此,若中间步骤有输入,需要手动输入,再运行到目标行
- 5) 按行执行:执行到下一行代码并停止运行
- 6) Step into: 进入到函数中单步执行
- 7) Step out: 从函数中跳出
- 8) 停止调试:整个程序的执行被终止,中途结束调试一定要点该按钮,不能直接关控制台



- 9) 监视区从 (Watches): 在调试窗口勾选,调试过程中监视变量的变化一边使用上面所说的调试方法,一边观察变量的值,从而找出自己代码中逻辑错误





### 三 分析与讨论

1. 讨论在安装 Code::Blocks 过程中的体验，是否遇到任何困难，以及如何解决这些问题；讨论 Code::Blocks 的默认配置是否适合初学者立刻开始编写和运行 C 程序。
2. 分析打开、新建项目的步骤是否简单明了，特别是对于初次使用者来说。
3. 反映编译过程中的体验，Code::Blocks 是否能够提供清晰的反馈信息来指导初学者解决编译错误。
4. Code::Blocks 与之前使用过的其他 IDE 的比较。

### 四 思考题

1. C 语言程序的构成与运行步骤。
2. 总结安装和配置 Code::Blocks 的步骤。
3. 讨论 Code::Blocks 作为一个开发工具的优点，适合什么样的项目开发？

### 五 注意事项

1. 在开始编程前，确认项目的配置设置正确，特别是编译器设置和链接器设置，避免因设置错误而导致编译失败。
2. 即使程序能够编译通过，也要关注编译器的警告信息。它们往往指向潜在的问题，可能在未来导致错误。
3. 如果编译器报告了许多错误，最好一次只修复一个错误，然后重新编译。因为一个小错误可能会产生连锁反应，导致多个错误信息。处理最上面的错误通常会解决后续的一些问题。

## 实验二 C 语言基础

### 一 实验目的

1. 了解 Code::Blocks 的基本操作方法，学习在环境中建立 C 程序。
2. 了解在 Code::Blocks 中如何编辑、编译、连接和运行一个 C 程序。
3. 通过运行简单的 C 程序，初步了解 C 语言程序的特点。
4. 掌握 C 语言数据类型，学会使用 C 的有关算术运算符。

### 二 实验内容

1. 打印出你的姓名学号。
2. 从键盘读取三个整数，输出它们中的最大值。
3. 写出以下程序的运行结果：

```
#include<stdio.h>

int main()
{
    float x;
    double y;
    x = 213.82631;
    y = 213.82631;
    printf("%-4.2f,%-6.2e\n", x, y);
    return 0;
}
```

4. 修改代码，以 float 类型分行打印三个 c 的值，分析代码中运算顺序。

```
#include<stdio.h>

int main()
{
    int a=10;
    float b=3.0;
    double c;
    c=a/b;
    c=a/(double)b;
    c=a+a*b-(int)b/a+5.5;
    return 0;
}
```

5. 运行下列代码，写出每个表达式的具体运算过程解释。

```
#include<stdio.h>

int main()
{
    int a=5, b=10, c=5;
    int x;
    x = (a = c) && (b > a);
    printf("%d\n", x);
    x = (a == b) || (a < b);
    printf("%d\n", x);
    x = !(a == b);
    printf("%d\n", x);
    return 0;
}
```

### 三 分析与讨论

1. 对于每个实验，记录操作过程和源代码。
2. 调试过程中出现的错误及修改情况。
3. 实验结果。
4. 总结 C 语言编程特点，程序的基本结构和编程的思想。

### 四 思考题

1. 探讨在不同数据类型（如 int、float、char）间进行运算时，如何进行类型转换，以及这些转换对计算结果可能产生的影响。
2. 哪些运算符会返回布尔值？列举例子说明
3. 括号在 2.5 中如何影响最终的实验结果？

### 五 注意事项

1. 了解运算符的优先级顺序，必要时使用括号来改变执行顺序，以避免逻辑错误。
2. 明确不同数据类型的区别，选择合适的数据类型来确保程序正确运行。
3. 注意类型溢出，理解不同数据类型的范围，避免溢出。
4. 使用清晰的命名、适当的注释和一致的代码格式，使代码易于理解和维护。

## 实验三 程序控制结构——分支

### 一 实验目的

- 1、掌握 if 语句的使用。
- 2、掌握 if...else 语句的使用。
- 3、掌握 if...else 语句的嵌套使用。
- 4、掌握 switch 语句的使用。

### 二 实验内容

- 1、使用 if 语句编写程序实现以下功能：从键盘任意输入一个 30 以内的正整数，先后自动判断是否为偶数、是否为质数、是否为 2 的 n 次方（是输出结论，否不用输出）。
- 2、使用 if...else 语句制作一段文字冒险游戏，至少包括 3 种结局。例如，程序输出：小明进山遇到了一只狼，他选择 1、拿起木棍打狼；2 逃跑。用户输入：1。程序输出：打赢了，小明选择 1、剥下狼皮；2、继续进山。
- 3、使用 switch 语句，制作一款抽奖系统，根据用户的输入判断奖金额度，至少分为特等奖、一等奖、二等奖、三等奖、参与奖，抽奖规则自定。

### 三 分析与讨论

1. 对于每个实验，记录操作过程和源代码。
2. 调试过程中出现的错误及修改情况。
3. 实验结果。
4. 总结选择控制语句的使用特点及其场景。

### 四 思考题

1. 思考 if 和 switch 的区别。
2. 思考 switch 语句中 case 所起到的实际作用。
3. 思考在现实编程过程中，选择控制语句的应用场合，举例说明。

### 五 注意事项

1. 注意 if 语句中（），{} 的用法。
2. 注意 switch 语句中 {} 的分布，case 的使用规范，以及 break 的用法。

## 实验四 程序控制结构——循环

### 一 实验目的

- 1、掌握 for 语句的使用。
- 2、掌握 while 语句的使用。
- 3、掌握 do...while 语句的使用。

### 二 实验内容

- 1、使用 for 语句输出\*号构成一个几何形状，可以是方形、菱形、圆形、心形等。

例如：

```
  *  
 ***  
*****  
 ***  
  *
```

- 2、使用 while 语句，找出小于你学号后 5 位的最大素数。
- 3、使用 do...while 语句，设计一款密码锁，连续输入 2 次正确密码则开锁并显示 1 中的绘制的图像，连续输入 5 次错误密码则禁止输入，程序退出。

### 三 分析与讨论

1. 对于每个实验，记录操作过程和源代码。
2. 调试过程中出现的错误及修改情况。
3. 实验结果。
4. 总结循环控制语句的使用特点及其场景。

### 四 思考题

1. 思考在现实程序中，循环控制语句所扮演的角色。
2. 思考那些程序功能是用循环控制语句编写的。
3. 思考 for、while、do...while 分别在什么场景下使用。

### 五 注意事项

1. 注意各个语句使用的语法规则。
2. 注意设置循环终止条件，避免陷入死循环。

## 实验五 数组：一维数组

### 一 实验目的

1. 熟悉一维数组和二维数组的定义、初始化及引用。
2. 掌握引用数组元素的方法。
3. 掌握字符串在内存中的存储方式。
4. 熟练掌握字符串的处理方法。

### 二 实验内容

1. 编写程序:使用筛选法求 100 之内的素数, 输出时每行最多 10 个数
2. 编写程序: 使用选择排序法对 5 个整数按照从小到大的顺序进行排序。
3. 编写程序:输入一行英文字符, 统计其单词总数(单词间以空格分隔)。例如, 输入内容为 “I am a boy.” 时, 应输出 “There are 4 words in the line” 。
4. 编写程序:删除字符串中指定的字符并输出。例如, 字符串是 “hello”, 要求删除字符 e, 输出 “hllo” 。

### 三 分析与讨论

1. 对于每个实验, 记录操作过程和源代码。
2. 调试过程中出现的错误及修改情况。
3. 实验结果。
4. 理解数组的索引

### 四 思考题

1. 根据题意填空, 将程序补充完整, 然后上机验证。下面程序的功能是输出数组 s 中最大元素的下标:

```
# include <stdio.h>

int main()
{
    int k, p;
    int s[ ]={1, -9, 7, 2, -10., 3};
    for (p=0 , k=p; p<6; p++)
        if(    )
            k=p;
    printf("%d \n", k);
    return 0;
```

```
}
```

2. 根据题意填空，将程序补充完整，然后上机验证。下面程序的功能为求矩阵主、次对角线元素之和：

```
#include <stdio.h>

Int main()
{
    static int a[ ][3]={9, 7, 5, 3, 1, 2, 4, 6, 8};
    int i, j, s1=0, s2=0;
    for(i=0; i<3; i++)
        for(j=0; j<3; j++)
        {
            If(          ) s1=s1+a[i][j];
            If(          ) s2=s2+a[i][j];
        }
    printf("%d\n%d\n", s1, s2);
    return 0;
}
```

## 五 注意事项

1. 定义数组时用的 `a[3][4]`和引用元素时用的 `a[3][4]`是完全不同的。前者 `a[3][4]`用来定义数组的维数和各维的大小，后者 `a[3][4]`中的 3 和 4 分别代表数组元素的行下标和列下标，此时 `a[3][4]`是一个数组元素。
2. 在使用字符数组的过程中，接收字符串时用了取地址运算符，如 “`char str[20]; scanf("%s", &str);`”。这样的写法是错误的，由于数组名本身就代表地址,所以不应再加 “&”。

## 实验六 数组：二维数组

### 一 实验目的

1. 掌握二维数组的定义、赋值及输入输出的方法。
2. 掌握与二维数组有关的算法如查找、矩阵转置等。
3. 掌握在程序设计中使用数组的方法。数组是非常重要的数据类型，循环中使用数组能更好地发挥循环的作用，有些问题不使用数组难以实现。
4. 掌握在 Code::Blocks 环境下上机调试二维数组程序的方法，并对结果进行分析。

### 二 实验内容

1. 二维数组的初始化，即给二维数组的各个元素赋初值。下面的几个程序都能为数组元素赋值，请输入程序并运行。

(1) 在定义数组的同时对数组元素分行初始化。

```
/*二维数组的初始化(分行)*/  
#include "stdio.h"  
void main()  
{ int i,j,a[2][3]={1,2,3},{4,5,6}};  
  for(i=0; i<2; i++)  
  { for(j=0; j<3; j++)  
    printf("%d ",a[i][j]);  
    printf("\n");  
  }  
}
```

(2) 不分行的初始化。把{ }中的数据依次赋值给数组的各个元素。

```
/*二维数组的初始化(不分行)*/  
#include "stdio.h"  
void main()  
{ int i,j,a[2][3]={1,2,3,4,5,6};  
  for(i=0;i<2;i++)  
  { for(j=0;j<3;j++)  
    printf("%d ",a[i][j]);  
    printf("\n");  
  }  
}
```

(3) 为部分数组元素初始化，如：数组定义语句为：int i,j,a[2][3]={1,2},{4}。

(4) 可以省略第一维的定义，但不能省略第二维的定义，如：int a[ ][3]={1,2,3,4,5,6}。

2. 求一个  $4 \times 4$  矩阵的主对角线元素之和，填空并运行程序。

(1) 定义一个 4 行 4 列的二维数组 a。

(2) 可利用双重循环的嵌套为该二维数组的各个数组元素赋值，一般格式为：

```
for(i=0; i<4; i++)
    for(j=0; j<4; j++)
        scanf("%d",&a[i][j]);
```

(3) 用循环求和，并注意矩阵对角上线元素的特征是：行下标和列下标相同。

(4) 输出对角线元素之和。

/\*求一个 4×4 矩阵的主对角线元素之和\*/

```
#include "stdio.h"
void main( )
{
    int a[4][4]={ {1,2,3,4},{5,6,7,8},{3,9,10,2},{4,2,9,6}};
    int i,sum=0;
    for(i=0; i<4; i++)
        _____; /*把对角线元素的和放在变量 sum 中 */
    printf("sum=%d\n",sum); /*输出对角线元素的和*/
}
```

3. 以下程序是查找二维数组 a 的最大元素及其下标，填空并运行程序。

/\*求二维数组中元素的最大值及其下标\*/

```
#include "stdio.h"
void main()
{
    int a[4][4]={ {1,2,3,4},{3,4,5,6},{5,6,7,8},{7,8,9,10}};
    int i,j,max,l,c; /* max 存放最大值，l、c 分别存放行和列的下标 */
    max=a[0][0];
    for(i=0; i<4; i++)
        for(j=0; j<4; j++)
            if(max<a[i][j]) {_____;l=i; c=j;}
    printf("max=%d,l=%d,c=%d%\n",max,l,c);
}
```

4. 打印出以下的杨辉三角（要求打印出 10 行）

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1
1 9 36 84 126 126 84 36 9 1
```

(1) 杨辉三角的特点是：第 1 列和对角线上的元素为 1，其它各元素的值都是上一行上一列元素和上一行前一列元素之和。

(2) 定义一个 10×10 的二维数组 a。

(3) 先用一个单层循环为第一列和对角线上的元素赋值。如：

```
for(i=0; i<10; i++) { a[i][i]=1;a[i][0]=1; }
```

(4) 再用一个双重循环嵌套为其它元素赋值。

```
for(_____; i<10; i++)  
    for(_____; _____; j++)  
        a[i][j]=a[i-1][j-1]+a[i-1][j];
```

想一想，划横线的地方应当填入什么内容？

(5) 计算之后再用一个双重循环嵌套输出杨辉三角。

5. 统计 3 个学生，每个学生 4 门课程的考试成绩，要求输出每个学生的总成绩，每个学生的平均成绩，3 个学生的总平均成绩。填空并运行程序。

```
/*学生成绩处理 */  
#include "stdio.h"  
void main( )  
{ int stu[3][4],i,j,t[3];  
  float sum=0,a[3];  
  for(i=0; i<3; i++) /* 输入三个学生的 4 门课程考试成绩 */  
      for(j=0; j<4; j++)  
          scanf("%d",&stu[i][j]);  
  for(i=0; i<3; i++)  
  { t[i]=0;  
    for(j=0; j<4; j++)  
    { sum+=stu[i][j]; /* sum 存放三个学生的 4 门课程总成绩 */  
      t[i]+=stu[i][j]; /* t[i]存放第 i 个学生的 4 门课程成绩 */  
    }  
    printf("%-6d",t[i]); /* 输出第 i 个学生的总成绩 */  
    _____;  
    printf("%-6.2f\n",a[i]); /* a[i]存放第 i 个学生的 4 门课程平均成绩 */  
  }  
  printf("average = %.2f\n",sum/12.0);  
}
```

6. 下面程序的功能是实现方阵（如：3 行 3 列）的转置（即行列互换）

如：如果原来的矩阵：

```
100  200  300  
400  500  600  
700  800  900
```

程序输出应为：

```
100  400  700  
200  500  800  
300  600  900
```

填空并运行程序。

```
/*矩阵转置*/  
#include "stdio.h"  
void main( )
```

```

{ int i,j,temp;
  int array[3][3]={ {100,200,300},{400,500,600},{700,800,900}};
  printf("\n 原来的矩阵为: \n");
  for(i=0; i<3; i++)
    { for(j=0; j<____; j++) printf("%7d",array[i][j]);
      printf("\n");}
  for(i=0; i<3; i++)
    { for(j=0; ____; j++)
      { temp=array[i][j];array[i][j]=array[j][i];array[j][i]=temp;} }
  printf("\n 转置后的矩阵为: \n");
  for(i=0; i<3; i++)
    { for(j=0; j<3; j++) printf("%7d",array[i][j]);
      _____}      /*输出一行后要换行*/
    }
}

```

### 三 分析与讨论

1. 对于每个实验，记录操作过程和源代码。
2. 调试过程中出现的错误及修改情况。
3. 实验结果。

### 四 思考题

1. 实验内容 1 中，给二维数组的初始化赋初值的四种定义方式有何异同？
2. 实验内容 4 中，最后用一个双重循环嵌套输出杨辉三角时，各层循环变量的初值和终值怎样确定？
3. 实验内容 6 中的矩阵转置能运用于非方阵吗？若不能，改采用何种方式实现非方阵的矩阵转置？

### 五 注意事项

1. C 规定，二维数组的行和列的下标都是从 0 开始的。
2. 要对二维数组的多个数组元素赋值，应当使用循环语句并在循环过程中，使数组元素的下标变化。可用下面的方法为所有数组元素赋值：

```

int i,j,a[3][3];
for(i=0; i<3; i++)
  for(j=0; j<3; j++)
    scanf("%d",&a[i][j]);

```

## 实验七 数组：字符数组

### 一 实验目的

1. 掌握字符数组的定义、初始化和应用。
2. 掌握字符串处理函数的使用。

### 二 实验内容

1. 输入下面的程序并运行，观察程序运行的结果，并分析原因（注意程序第 2 行中有些单引号之间是空格）。

```
/*字符数组的输出*/
#include "stdio.h"
void main( )
{ char a[10]={ 'T', ' ', 'a', 'm', ' ', 'a', ' ', 'b', 'o', 'y'};
printf("%s\n",a);
}
```

2. 按照要求编写程序：有一行文字，不超过 80 个字符，分别统计出其中英文大写字母、小写字母、数字、空格、及其它字符的个数。

(1) 定义一个一维字符数组。

(2) 定义 5 个计数器使用并赋初值。

(3) 用 scanf 函数或 gets 函数为字符数组赋一个字符串。

(4) 在循环中对字符数组的每个元素进行判断，相应的计数器加 1。注意循环控制的条件和进行判断的条件怎样设置。

(5) 循环结束后输出各计数器的值。

```
/*统计字符的个数*/
#include "stdio.h"
void main( )
{ char a[80]; //存放字符
char str;
int i;
int b[5]={0}; //存放统计的个数
gets(a);

for(i=0;(str=a[i])!='\0';i++)
{
if(str>='a'&&str<='z') //统计小写字母个数
b[0]++;
else if(str>='A'&&str<='Z') //统计大写字母个数
b[1]++;
}
```

```

        else if(str>='0'&&str<='9') //统计数字个数
            b[2]++;
        else if(str==' ') //统计空格个数
            b[3]++;
        else b[4]++; //统计其他字符个数
    }
    printf("小写字母:%d\n",b[0]);
    printf("大写字母:%d\n",b[1]);
    printf("数字:%d\n",b[2]);
    printf("空格:%d\n",b[3]);
    printf("其他字符:%d\n",b[4]);
}

```

3. 下面程序的功能是实现将两个字符串连接起来并输出结果，注意不使用 `strcat` 函数。请填空并运行程序。

(1) 定义两个一维字符型数组 `str1`、`str2` 和两个循环变量。

(2) 为两个字符数组输入两个字符串（可使用 `scanf` 函数或 `gets` 函数整体赋值，要注意 `scanf` 和 `gets` 函数的区别，在对字符串赋值时，`scanf` 函数不能出现空格）。

(3) 确定字符数组 `str1` 结束的位置。

(4) 再将字符数组 `str2` 中的内容连接到字符数组 `str1` 的后面。

(5) 为字符数组 `str1` 赋字符串结束的标志 `'\0'`。

(6) 输出连接后的字符数组 `str1`。

```

/*字符串连接*/
#include "stdio.h"
void main( )
{
    char str1[100],str2[100];
    int i=0,j=0;
    printf("please input the string1:");
    scanf("%s",str1);
    printf("please input the string2:");
    gets(str2);
    for(i=0; str1[i]!='\0'; i++);    /*注意,此处空语句不可少*/
    for(j=0;str2[j]!='\0';j++)
    { str1[i]=str2[j];
      i++;
    }
    _____;    /*给出新的字符串的结束符*/
    printf("the catenated string is %s",str1);
}

```

4. 下面程序的功能是实现将一个字符串中的所有大写字母转换为小写字母并输出，请补充完整。

例如：当字符串为"This Is a c Program"

输出： "this is a c program"

/\*字符串中的大写字母转为小写字母\*/

```
#include "stdio.h"
void main( )
{
    char str[80]="This Is a c Program";
    int i;
    printf("String is: %s\n", str);
    for (i=0; str[i]!='\0'; i++)
        if (str[i]>='A' && str[i]<='Z')
            _____; /*将大写字母转换成小写字母*/
    printf("Result is: %s\n",str);
}
```

### 三 分析与讨论

1. 对于每个实验，记录操作过程和源代码。
2. 调试过程中出现的错误及修改情况。
3. 实验结果。

### 四 思考题

1. 实验内容 1 中，将字符数组 a 的大小改为 11，再运行程序，并将结果与修改前的结果进行比较，分析原因。
2. 实验内容 2 中，如果是对一篇英文文章进行统计,又该怎么编程呢？文章的行数和每行字数可以自己来设。提示：对文章的内容要用二维字符数组来存储。
3. 实验内容 3 中，如果使用 strcat 函数该如何实现将字符串 2 连接到字符串 1 的后面并输出？
4. 实验内容 4 中，如果将字符串中的所有小写字母转换为大写字母，又将如何修改程序？

### 五 注意事项

1. 注意 C 语言中字符串是作为一维数组存放在内存中的，且系统对字符串常量自动加上一个 ‘\0’ 作为结束符，所以在定义一个字符数组并初始化时要注意数组的长度。注意用 scanf 函数对字符数组整体赋值的形式。

# 实验八 指针

## 一 实验目的

1. 通过实验加深理解指针概念，掌握指针变量的定义与应用；
2. 学会使用数组的指针和指向数组的指针变量；
3. 学会使用字符串指针和指向字符串的指针变量；
4. 了解指向指针的指针的概念及其使用方法。

## 二 实验内容

1. 写出以下程序的运行结果

```
#include<stdio.h>

main()
{
    int a=100,*p1;
    float b=12.56,*p2;
    char c='A',*p3;
    char s[]="ABCDEF",*p4;

    p1=&a;
    p2=&b;
    p3=&c;
    p4=&s[0]; //或 p4=s;
    printf("%d %d\n",a,*p1);
    printf("%f %f\n",b,*p2);
    printf("%c %c\n",c,*p3);
    printf("%c %c\n",*p4,*(p4+2));
}
```

2. 输入 3 个整数，按由小到大的顺序输出。
3. 有一个 3\*4 的矩阵，编写一程序利用指针变量实现矩阵的转置。
4. 运行下列代码，并解释其功能

```
#include<stdio.h>

main()
{
    int a[5]={1,2,3,4,5};
    int *p;
```

```
p=&a[0]; //或 p=a;
printf("%d %d\n",*p,*(p+2));
printf("%d %d %d\n",a[0],a[2],*(a+2));
}
```

5. 输入一行文字，统计大写字母、小写字母、空格、数字字符及其他字符的个数。

### 三 分析与讨论

1. 对于每个实验，记录操作过程和源代码。
2. 调试过程中出现的错误及修改情况。
3. 实验结果。
4. 总结指针部分的相关原理、作用和基本用法。

### 四 思考题

1. 分析总结对于不同的数据类型，如整数和字符串，指针变量的用法有何不同？
2. 怎样利用指针求一个字符串的长度？
3. 了解指向指针的指针的基本原理和用法。

## 实验九 函数——函数的定义和调用、函数间的数据传输

### 一 实验目的

1. 掌握函数的定义、调用、参数传递;
2. 掌握函数嵌套调用和递归调用的方法;
3. 掌握全局变量和局部变量、动态变量、静态变量的概念和使用方法。

### 二 实验内容

1. 求 2 个数中较大者。
2. 编写函数，计算两个整数的最大公约数和最小公倍数。
3. 写出以下程序的运行结果。

```
#include<stdio.h>
```

```
void fun(int i,int j)
void main()
{
void fun(int,int);
int i,j,k,l,x,y;
i=2;j=3;k=4;l=5;
x=6;y=7;
fun(i,j);
printf("i=%d, j=%d\n",i,j);
printf("x=%d, y=%d\n",x,y);
fun(k,l);
}
void fun(int i,int j)
{
int x,y;
x=8;y=9;
i=i+1;
j=j+1;
printf("i=%d, j=%d\n",i,j);
printf("x=%d, y=%d\n",x,y);
}
```

4. 从键盘输入 10 名学生的成绩，显示其中的最低分、最高分及平均成绩。
5. 编写一函数，实现两个字符串的复制。

### 三 分析与讨论

1. 对于每个实验，记录操作过程和源代码。
2. 调试过程中出现的错误及修改情况。
3. 实验结果。
4. 总结函数的定义、调用、参数传递等。

### 四 思考题

1. 对于多个数据如何求最大公约数和最小公倍数？
2. 写一个函数，使其能将一个二维数组（5×3）中的数据进行行列互换。
3. 写出下面程序的运行结果，理解动态局部变量和静态局部变量的区别。

```
#include <stdio.h>

int fun(int n)
{
    static int b=2;
    int m;
    b*=2;
    m=b+n;
    return m;
}

int main()
{
    int a=2, i, k;
    for(i=1; i<=3; i++)
    {
        k=fun(i);
        printf("k=%d\n", k);
    }
    return 0;
}
```

如果将 fun()函数中的语句“static int b=2;”改为“int b=2;”,程序运行的结果又会怎样呢？

4. 写出调用函数 f(27)的输出结果:

```
void f(int n)
{
    if(n<5) printf("%d", n);
    else
    {
```

```

        printf("%d", n%5);
        f(n/5);
    }
}

```

5. 编写程序，输入一个正实数  $x$  和一个正整数  $n$ ，求下列算式的值。

$$\frac{x - x^2}{2!} + \frac{x^3}{3!} - \frac{x^4}{4!} + \frac{x^{(-1)^{n-1}}}{n!}$$

要求: 定义和调用两个函数，`fact(n)`计算  $n$  的阶乘，`mypow(x,n)`计算  $x$  的  $n$  次幂(即 $x^n$ )，两个函数的返回值类型都是 `double`。

## 五 注意事项

1. 实参与形参的类型必须一致。函数被调用前，形参不占用内存；函数调用结束后，形参所占用的内存也将被收回。实参和形参占用不同的内存单元，即使同名也互不影响。
2. 由于在递归函数中，存在着调用自身的过程，控制将反复进入自身函数体执行，因此在函数体中必须设置终止条件，当条件成立时，终止调用自身，并使控制逐步返回主调函数。
3. 建议尽量少使用全局变量。因为各个函数中，都能修改全局变量的值，人们往往难以清楚地判断出每个瞬时各个全局变量的值，从而降低程序的清晰性。

# 实验十 函数——应用程序

## 一 实验目的

1. 掌握复杂函数的编写；
2. 掌握如何利用函数实现指定功能；
3. 利用函数的编写、调用等实现特定的应用程序。

## 二 实验内容

1. 编写求圆周长、圆面积、球体积的程序。要求在主程序中输入半径、输出圆周长、圆面积、球体积，用函数分别实现求圆周长、圆面积、球体积。
2. 写一个程序，从键盘输入 5 个正整数，然后求出它们的最小公倍数。
3. 编写函数将两个两位数的正整数  $a$ 、 $b$  合并形成一个整数放在  $c$  中。合并的方式是：将  $a$  数的十位和个位数依次放在  $c$  数的千位和十位上， $b$  数的十位和个位数依次放在  $c$  数的个位和百位上。例如，当  $a=45$ ， $b=12$ 。调用该函数后， $c=4251$ 。要求：在主函数中输入  $a$ 、 $b$ ，输出  $c$ 。
4. 用递归方法求  $n!$
5. 设有一个 3 位数，将它的百、十和个位 3 个单一数各自求立方，然后加起来，正好等于这个 3 位数，如  $153=1+125+27$ 。写一个函数，找出所有满足条件的数。

## 三 分析与讨论

1. 对于每个实验，记录操作过程和源代码。
2. 调试过程中出现的错误及修改情况。
3. 实验结果。
4. 总结复杂函数的用法，掌握特定应用程序的编写。

## 四 思考题

1. 如果一个数正好是它的所有约数（除了它本身以外）的和，此数称完备数，如：6，它的约数有 1、2、3，并且  $1+2+3=6$ 。求出 30 000 以内所有的完备数，并显示输出。
2. 如果有两个数，每一个数它的所有约数（除了它本身以外）的和正好等于对方，则称这两个数为互满数，求出 30 000 以内所有的互满数，并显示输出。